1. Riepilogo degli Obiettivi

Sostituzione del “motore” interno nel progetto già esistente ARMulator con Unicorn, al fine di implementare ARMv7.

2. Struttura repository

- ***interface:*** contiene *static*, *tests* e il *simulator.html.*
  → *Static:* contiene i file per la parte statica (tutto ciò che non presenta funzione della GUI) e i file .js per renderla dinamica.
  → *Tests:* è presente tutto ciò che riguarda la formattazione del testo.
  → *Simulator.html*: contiene la pagina web.

- **pdoc-docs:** contiene tutta la documentazione necessaria a capire questo progetto. La documentazione HTML è generata automaticamente da **pdoc**, uno strumento che legge il codice Python e crea una pagina web leggibile con la descrizione di ogni *funzione* e *classe*.

- **profile data:** contiene i *cookies* e la *cache.*

- **samples:** contiene degli esempi di codice ARM.

- **simulator.ops:** contiene la gestione manuale e la creazione degli oggetti per ogni singola operazione ARM che la CPU esegue. Verrà eliminata e sostituita con **Unicorn**.

- **tests**: contiene dei test per verificare il funzionamento di *assembler* e *bytecode* confrontando dei codici ARM già esistenti.

- **translation:** contiene *dictionary,* che vengono utilizzati per la traduzione in 3 lingue (Italiano, Inglese e Francese).

- **utils**: contiene gensymbs.py. Questo script è utilizzato per generare una stringa **Regex (Espressione Regolare)** che include tutte le possibili combinazioni di istruzioni per il linguaggio assembly **ARM**.

- **.gitignore**: dice a *git* quali file ignorare e non includere nella repository.

- **assembler.py:** trasforma il codice sorgente *assembly* in *bytecode* binario che una CPU (o un simulatore) può effettivamente eseguire. Spiega come gestire la memoria e le sezioni e possiede due modalità di mappatura: per simulazione e per test. Qui vengono esplicitate anche le dimensioni di *INTVEC* (vettore degli interrupt), *code*, *data*.

- **bytecodeinterpreter.py**: funge da ponte di comunicazione principale tra l'interfaccia utente (UI) e il core del simulatore ARM.

- **components.py:** gestisce tutta la memoria e come avviene la scrittura e la lettura dei registri.

- **main.py:** segue questo flusso: *file .txt → ASMparser → bytecode → BCInterpreter → esegui → leggi registri.*

- **mainweb.py:** fa 3 cose:
  1. *Server HTTP (Bottle)*→ serve la pagina web
  2. *Server WebSocket*→ comunicazione in tempo reale con il browser
  3. *Logica di simulazione*→ gestisce i comandi dell'utente

- **simulator.py:** ha 4 responsabilità:
  1. *Fetch*→ legge i bytes dall'indirizzo PC;
  2. *Decode*→ capisce che istruzione è;
  3. *Execute*→ esegue l'istruzione;
  4. *Gestione*→ *breakpoints, interrupt, history, assertion.*

- **history.py**: gestisce il r*everse debugging* (*step-back*). È collegato a *Memory* e *Registers* tramite *signalChange( )*. Unicorn deve rispettare questo meccanismo.

- **settings.py:** contiene parametri globali come *PCbehavior* = "+8", che è fondamentale per Unicorn, perché il **PC** nel simulatore vale sempre come *PC_reale + 8.*

- **stateManager.py:** *singleton* che gestisce lingua e porte del server. Usato ovunque tramite appState.getT(index) per le traduzioni.

3. Difficoltà incontrate

- Capire come erano collegati i vari file all'interno del progetto.
- Installazione del progetto da developer.

4. Test e Risultati

- ***FailedTry.py*** è stato un tentativo di integrazione non riuscito.

- ***min_engine.py***, il motore stand-alone, che, per il momento, prende in input un codice in ARM, che è interpretabile da ***Keystone*** (libreria che stiamo usando insieme a Unicorn).

5. Tabella di marcia

- **simulator.ops** va eliminato e sostituito con *Unicorn Engine*, un framework di emulazione basato su *QEMU*. Questo velocizza l'interpretazione delle esecuzioni, poiché utilizza la compilazione *JIT (Just - In – Time).*

- **simulator.py** potrebbe essere riciclato installando Unicorn Engine al suo interno.